Reviewer Pack — Minimal Rank of Free Probability Spaces Bounds Randomized Co…

Entry 4e1a3730c35d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 17:01:07 UTC

Minimal Rank of Free Probability Spaces Bounds Randomized Communication Complexity for Disjointness

Entry ID: `4e1a3730c35d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-28 17:01:07 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Communication Complexity (Disjointness)

Statement:

> {'p1': 'Let M be a matrix representing the free probability space of an n-dimensional random variable. Then the minimal rank of M, denoted as r(M), lower bounds the randomized communication complexity of Disjointness on n bits with a factor of at least 2.', 'p2': 'Equivalently, if there exists a communication protocol for Disjointness that uses fewer than 2 * r(M) bits in expectation, then M contains a non-trivial invariant under free convolution.', 'p3': 'Thus, τ(DISJ_n) = Ω(n) implies r(M) ≥ n/2.'}

Rationale (proposer's reasoning):

> {'p1': 'Free probability spaces are highly non-commutative and can encode complex correlations between variables. Their minimal rank measures the complexity of these correlations.', 'p2': 'Communication complexity for Disjointness has been shown to be sensitive to the structure of the underlying random variables, suggesting that the minimal rank of their associated free probability space could serve as a lower bound.', 'p3': 'The conjecture leverages the algebraic nature of free probability theory and its potential connection to non-commutative geometry, which could provide new insights into communication complexity.'}

Taxonomy category: `COMM_DISJ` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `a73f07f4e520ba06`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if at least 70% of the 30 random seeds show a correlation coefficient between r(M) and communication complexity greater than or equal to 0.7, with no seed demonstrating a communication complexity less than 2 * r(M).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "free probability theory" AND "communication complexity" AND "randomized disjointness" - "minimal rank" free probability AND communication complexity bounds AND disjointness" - "Invariant under free convolution" AND "communication protocol" AND "disjointness problem"

Top relevant hits considered: - [http://arxiv.org/abs/2603.19490v1] Communication Complexity of Disjointness under Product Distributions - [http://arxiv.org/abs/1204.6522v3] Formal Groups, Witt vectors and Free Probability - [http://arxiv.org/abs/1304.1217v1] On the communication complexity of sparse set disjointness and exists-equal problems

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 10  # Start with a small size and increase if needed

    def generate_matrix(n):
        M = [[random.uniform(-1, 1) for _ in range(n)] for _ in range(n)]
        return M

    def matrix_rank(M):
        m, n = len(M), len(M[0])
        rank = 0
        A = [row[:] for row in M]

        for i in range(min(m, n)):
            if A[i][i] == 0:
                swap_found = False
                for j in range(i + 1, m):
                    if A[j][i] != 0:
                        A[i], A[j] = A[j], A[i]
                        swap_found = True
```

```
                    break
            if not swap_found:
                continue

        pivot = A[i][i]
        for j in range(n):
            A[i][j] /= pivot

        for j in range(m):
            if j != i:
                factor = A[j][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]

        rank += 1

    return rank

def simulate_disjointness(n):
    bits = [random.choice([0, 1]) for _ in range(n)]
    communication_cost = n  # Simplified model
    return communication_cost

M = generate_matrix(n)
r_M = matrix_rank(M)
comm_complexity = simulate_disjointness(n)

if comm_complexity < 2 * r_M:
    return {
        "metric_name": "communication_complexity",
        "metric_value": comm_complexity,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"r(M)={r_M}, comm_complexity={comm_complexity}"
    }

return {
    "metric_name": "communication_complexity",
    "metric_value": comm_complexity,
    "instances_tested": 1,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    total_comm_complexity = 0
    total_instances = 0
    support_count = 0

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
```

```
        total_comm_complexity += trial_result["metric_value"]
        total_instances += trial_result["instances_tested"]
        if trial_result["conjecture_holds"]:
            support_count += 1

    mean_comm_complexity = total_comm_complexity / total_instances
    std_comm_complexity = math.sqrt(sum((x - mean_comm_complexity) ** 2 for x in [trial_result["me
    support_fraction = support_count / len(seeds)

    if support_fraction >= 0.7:
        print(f"RESULT: SUPPORTED mean={mean_comm_complexity} std={std_comm_complexity} support_fr
    else:
        first_failing_seed = next(seed for seed, trial_result in zip(seeds, results) if not trial_
        print(f"RESULT: FALSIFIED counterexample='r(M)<2*comm_complexity' first_failing_seed={firs
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
'communication_complexity', 'metric_value': 10, 'instances_tested': 1, 'conjecture_holds': False, 'c
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 10, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 10, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 10, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 10, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 10, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 10, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 10, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 10, 'instances_tested': 1, 'conje

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4fea49f.py", line 102, in <module>
    std_comm_complexity = math.sqrt(sum((x - mean_comm_complexity) ** 2 for x in [trial_result["metric_
                                    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
ZeroDivisionError: division by zero
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test results demonstrate that the conjecture does not hold for at least one instance where r(M) = 10 and communication complexity is also 10, whic | next: Investigate cases where r(M) is significantly larger than the communication complexity to understand why this counterexample occurs. Consider whether there are limitations in the test setup or if the conjecture needs to be revised.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12799
2	preregistration	ollama_remote	glm4:latest	0	0	5737
3	novelty	ollama_remote	glm4:latest	0	0	4732
4	novelty	ollama_remote	glm4:latest	0	0	5704
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23013
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8328
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10220
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9507
9	judge	ollama_remote	glm4:latest	0	0	11561

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 91602 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/4e1a3730c35d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4e1a3730c35d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/4e1a3730c35d.tar.gz` (if generated)